

CSCE 40103: INTRO TO AI FOR SECURITY

LAB 2: EMAIL TEXT CLASSIFICATION

Introduction

In this lab, you will build a guided text classification workflow using the **SMS Spam Collection Dataset**. You will inspect the dataset, analyze class imbalance, convert raw text into TF-IDF features, train Naive Bayes and Linear SVM classifiers, evaluate the models, and inspect misclassified messages from an email-filtering perspective. This lab is focused on classical text classification.

Dataset

- Dataset file: *SMS_Spam_Collection_Dataset.csv*
- Text column: *text*
- Target column: *label*
- Class labels: *ham* and *spam*
- Positive class for evaluation: *spam*
- Rows: 5,574 messages
- Class distribution: 4,827 ham messages and 747 spam messages

Instructions

- Create a Jupyter Notebook named `Lab2-LastName.ipynb`.
- Place *SMS_Spam_Collection_Dataset.csv* in the same folder as your notebook.
- Use markdown headings for each lab step.
- Run every code cell before submitting.
- Include your name, date, course name, and lab number at the top of the notebook.
- Write your own code for each step. Do not paste a full AI-generated notebook.
- If you use AI tools for debugging or explanation, include a short **AI Use Statement** at the end.

Learning Objectives

- Load and inspect a labeled text dataset.
- Explain why raw text must be converted into numeric features.
- Create and interpret simple text-length features for EDA.
- Use TF-IDF to represent message text for classification.
- Train and evaluate Naive Bayes and Linear SVM classifiers.
- Interpret precision, recall, F1-score, and confusion matrix results.
- Inspect false positives and false negatives in an email-filtering context.
- Explain why dataset limitations matter for security claims.

Required Packages

Use the following packages and tools. You may import them in separate cells or one organized import cell.

- `pandas` and `numpy` for data handling
- `matplotlib` and `seaborn` for visualization
- `train_test_split` and `cross_validate` for evaluation workflow
- `TfidfVectorizer` for text feature extraction
- `Pipeline` for keeping vectorization and modeling together
- `MultinomialNB` for Naive Bayes classification
- `LinearSVC` for Linear SVM classification
- `classification_report`, `confusion_matrix`, and `ConfusionMatrixDisplay` for evaluation

Function and Method Parameters Used in This Lab

Function or Method	Important Parameters	Purpose
pd.read_csv()	filepath_or_buffer	Loads the CSV file.
df.head()	n=5	Displays the first five rows.
df.info()	No required parameter	Displays column names, data types, and non-null counts.
df.isnull().sum()	No required parameter	Counts missing values by column.
df.duplicated().sum()	No required parameter	Counts exact duplicate rows.
value_counts()	normalize	Computes class counts and percentages.
str.len()	No required parameter	Computes message length in characters.
str.split()	No required parameter	Splits text into words for approximate word count.
train_test_split()	X, y, test_size, stratify, random_state	Creates training and testing sets while preserving class proportions.
TfidfVectorizer()	stop_words, max_features, ngram_range, min_df	Converts text into TF-IDF feature vectors.
Pipeline()	steps	Chains TF-IDF and model training safely.
MultinomialNB()	alpha	Creates a Naive Bayes text classifier.
LinearSVC()	C, random_state	Creates a Linear SVM classifier.
cross_validate()	estimator, X, y, cv, scoring	Evaluates a model across folds using multiple metrics.
classification_report()	y_true, y_pred, zero_division	Displays precision, recall, F1-score, and support.
confusion_matrix()	y_true, y_pred	Computes false positives and false negatives.
ConfusionMatrixDisplay()	confusion_matrix, display_labels	Displays a confusion matrix figure.

Step 1: Create the Notebook

- Create a new Jupyter Notebook named:

Lab2-LastName.ipynb

- At the top of the notebook, add a markdown cell with your name, date, course name, and lab number.
- Create clear markdown headings for each step.

Required output: Notebook header and organized section headings.

Step 2: Load the Dataset

- Load *SMS_Spam_Collection_Dataset.csv* into a DataFrame.
- Display the first five rows and confirm that the dataset contains the columns `label` and `text`.

Required output: First five rows and column check.

Step 3: Inspect Dataset Size and Data Types

- Display the number of rows and columns.
- Display column names and data types.
- Briefly describe what each column means.

Required output: Dataset shape, data type summary, and a short column description.

Step 4: Check Missing Values and Duplicates

- Check missing values in label and text.
- Check exact duplicate rows and duplicate text messages.
- Briefly state whether duplicates should be removed before modeling.

Required output: Missing value summary, duplicate counts, and one short interpretation.

Step 5: Inspect the Target Variable

- Compute class counts for *ham* and *spam*.
- Compute class percentages.
- Create a class distribution plot.
- Explain whether the dataset is balanced or imbalanced.

Required output: Class count table, class percentage table, plot, and imbalance explanation.

Step 6: Create Simple Text Features for EDA

- Create a character-count feature from the `text` column.
- Create an approximate word-count feature from the `text` column.
- Compare the average and median length of ham and spam messages.

Required output: Summary table comparing ham and spam message lengths.

Step 7: Visualize Message Length by Class

- Create at least one plot comparing text length across *ham* and *spam*.
- Use character count or word count.
- Briefly explain whether *spam* messages appear longer, shorter, or similar to *ham* messages.

Required output: Length comparison plot and two to three sentence interpretation.

Step 8: Create `X` and `y`

- Use the `text` column as `X`.
- Use the `label` column as `y`.
- Set *spam* as the positive class in your interpretation.

Required output: Shapes of `X` and `y` plus a note identifying the positive class.

Step 9: Train-Test Split

- Split the data into training and testing sets.
- Use stratification so *ham* and *spam* proportions stay similar in both sets.
- Use a fixed `random_state` so results are reproducible.

Required output: Training/testing row counts and class proportions in each split.

Step 10: Build the Naive Bayes Pipeline

- Create a Pipeline with `TfidfVectorizer` followed by `MultinomialNB`.
- Use English stop-word removal or explain why you chose not to.
- Train the pipeline on the training data.

Required output: A trained Naive Bayes text classification pipeline.

Step 11: Evaluate Naive Bayes

- Predict labels for the test set.
- Display the classification report.
- Display the confusion matrix.
- Identify the number of false positives and false negatives.

Required output: Classification report, confusion matrix, false positive count, and false negative count.

Step 12: Build the Linear SVM Pipeline

- Create a Pipeline with `TfidfVectorizer` followed by `LinearSVC`.
- Use the same train-test split as Naive Bayes.
- Train the model on the training data.

Required output: A trained Linear SVM text classification pipeline.

Step 13: Evaluate Linear SVM

- Predict labels for the test set.
- Display the classification report.
- Display the confusion matrix.
- Identify false positives and false negatives.

Required output: Classification report, confusion matrix, false positive count, and false negative count.

Step 14: Create a Model Comparison Table

- Compare Naive Bayes and Linear SVM using accuracy, precision, recall, and F1-score.
- Do not choose the best model using accuracy alone.
- Briefly explain which model is better for catching spam and which model is safer for avoiding false alarms.

Required output: Metric comparison table and short interpretation.

Step 15: Use Stratified Cross-Validation

- Run stratified cross-validation for both models.
- Use accuracy, precision, recall, and F1 as scoring metrics.
- Compare cross-validation results with the train-test split results.

Required output: Cross-validation summary table for both models.

Step 16: Inspect Misclassified Messages

- Create a table of false positives: *ham* messages predicted as spam.
- Create a table of false negatives: spam messages predicted as *ham*.
- Show up to five examples from each category if available.
- Explain what patterns may have confused the model.

Required output: False positive and false negative examples with interpretation.

Step 17: Compare Model Behavior on Real Test Messages

- Select a small set of real test messages where the two models disagree, if any are available.
- For each selected message, show the true label, Naive Bayes prediction, Linear SVM prediction, and a shortened message preview.
- Explain which model decision seems more reasonable based on the message content and the true label.

Required output: A model-disagreement table or a note that no disagreements were found, plus a short interpretation.

Step 18: Final Reflection

Write a short reflection of approximately 150 to 250 words addressing the following questions:

- Which model would you choose for a university email warning system?
- Which error is more serious: spam marked as ham, or ham marked as spam?
- Why is this not a true phishing detector?
- What additional data would be needed to build a real phishing detector?

Required output: A short written reflection, about 150 to 250 words.

What to Submit

- Notebook file: Lab2-LastName.ipynb
- Submit only the notebook file unless instructed otherwise.
- The notebook must be fully executed before submission.
- All required outputs must be visible in the notebook.
- Include an AI Use Statement if AI tools were used.

Grading Rubric

Category	Points
Notebook setup and organization	10
Dataset loading, inspection, missing values, and duplicate analysis	15
Class distribution and imbalance interpretation	10
Text-length feature creation and EDA visualization	10
Correct train-test split with stratification	10
Naive Bayes pipeline and evaluation	15
Linear SVM pipeline and evaluation	15
Model comparison and cross-validation summary	10
Mistake inspection and final reflection	5